

RAPPORT D'ÉVALUATION APPSEC

Application web de gestion administrative et des frais scolaires

Audit, analyse des vulnérabilités, remédiation, validation et hardening

Type de document	Rapport technique PFE – Bachelor cybersécurité / cyberdéfense
Référentiels	OWASP Top 10, AppSec, défense en profondeur, gestion sécurisée des sessions
Portée	Frontend web, API backend, authentification, autorisation, sessions, intégration Firebase/Firestore
Statut	Version consolidée enrichie après intégration d'un nouveau constat sur la gestion des JWT
Confidentialité	Document à usage pédagogique et technique - diffusion restreinte
Date	Mars 2026
Auteur	Amine BAHOU / Anass Akker

Objet du rapport

Ce document consolide les vulnérabilités identifiées jusqu'à présent sur l'application cible et les reformule dans un format plus académique et plus exploitable pour un PFE de Bachelor. Il intègre une lecture orientée risque, architecture, causes racines, scénarios d'abus, plan de remédiation, validation post-correction et recommandations de sécurisation durable.

Par rapport à la version précédente du rapport, cette édition ajoute : un nouveau constat critique sur l'exposition d'un jeton de session JWT, une section d'analyse des causes racines, un mapping OWASP, une matrice avant / après remédiation, une stratégie de re-test, une section sur les contrôles de sécurité à implémenter et une annexe d'illustrations expurgées.

Table des matières

1. Résumé exécutif.....	3
2. Contexte, objectifs et périmètre	3
3. Méthodologie d'évaluation AppSec.....	4
4. Vue d'architecture et zones de confiance	4
Lecture sécurité de l'architecture :	5
5. Analyse des causes racines.....	5
6. Détail des constats techniques	6
F-01 - Privilege Escalation via Role Manipulation	6
Vulnérabilité de session : JWT exposé et structure faible	6
F-02 - Privilege Escalation in Registration Flow.....	8
F-03 - Missing Rate Limiting on Authentication Endpoint	9
F-04 - Missing Input Size Validation	10
F-05 - Weak Server-Side Security Architecture	11
F-06 - Firebase Attack Surface and Error Handling	12
F-07 - JWT exposé et structure de session à renforcer	13
F-09 - Divulgence d'informations techniques : exposition de la version du serveur web	14
7. Remédiation, validation et re-test.....	17
7.1. Modèle de validation par vulnérabilité.....	17
7.2. Tableau de synthèse avant / après	17
7.3. Stratégie de re-test recommandée	17
8. Contrôles de sécurité à mettre en place	18
9. Mapping des constats avec l'OWASP Top 10	18
10. Feuille de route de remédiation priorisée.....	19
11. Limites de l'étude et perspectives	19
12. Protection des données personnelles et conformité RGPD	20
13. Conclusion générale.....	21

1. Résumé exécutif

Dans le cadre de ce projet de fin d'études en cybersécurité et cyberdéfense, une évaluation AppSec a été menée sur une application web de gestion administrative et des frais scolaires. L'objectif n'est pas uniquement d'identifier les vulnérabilités, mais également d'analyser leurs causes racines, d'en mesurer l'impact métier et de proposer des actions de sécurisation concrètes et vérifiables.

Les constats confirment une faiblesse structurelle au niveau des contrôles d'autorisation côté serveur et, désormais, un risque supplémentaire sur la gestion des sessions. La combinaison de défauts d'autorisation, d'une confiance excessive accordée au client et d'une exposition de jeton d'authentification augmente fortement le risque de compromission d'un compte privilégié et de prise de contrôle fonctionnelle de la plateforme.

ID	Niveau	Constat	Priorité	Statut
F-01	Critique	Manipulation du champ role lors de la mise à jour du profil	Immédiate	Confirmé
F-02	Critique	Attribution non autorisée de privilèges dans le flux d'inscription	Immédiate	Confirmé / à revalider après correctif
F-03	Élevé	Absence de rate limiting sur l'authentification	Très haute	Confirmé
F-04	Moyen	Validation insuffisante de la taille des entrées	Haute	Confirmé
F-05	Élevé	Contrôles de sécurité insuffisants côté serveur	Très haute	Confirmé
F-06	Moyen	Gestion des erreurs / surface Firebase à approfondir	Planifiée	Observation ciblée
F-07	Critique	JWT de session exposé et structure de session à renforcer	Immédiate	Nouveau constat

Conclusion de synthèse. Le risque dominant ne provient pas d'une attaque extrêmement sophistiquée, mais d'un défaut de mise en œuvre des mécanismes fondamentaux de sécurité applicative : contrôles d'accès côté serveur, gestion des rôles, politique de session robuste, validation des entrées et défense en profondeur. Tant que l'API accepte des attributs sensibles provenant du client ou qu'un jeton actif peut être réutilisé hors contrôle strict, la surface d'attaque reste critique.

2. Contexte, objectifs et périmètre

Le cahier des charges décrit une application de gestion des frais de scolarité et des documents administratifs, avec plusieurs profils métiers - administration, comptabilité, parents / tuteurs, étudiants et staff - s'appuyant sur une interface web, un backend applicatif et une base de données Firestore. Le système doit notamment gérer l'authentification, les rôles et permissions, les paiements, les documents et la journalisation.

Les objectifs de l'évaluation sont les suivants :

- identifier les vulnérabilités exploitables affectant l'authentification, l'autorisation, les sessions et la robustesse des endpoints ;
- analyser les causes racines techniques et organisationnelles ;

- prioriser les risques en fonction de leur impact métier et de leur exploitabilité ;
- proposer des correctifs concrets et des mécanismes de hardening ;
- définir une méthodologie de validation post-correction afin de démontrer la réduction effective de la surface d'attaque.

Périmètre couvert :

- interface web et composants exposés côté client ;
- requêtes API manipulées manuellement via proxy d'interception ;
- flux d'inscription, d'authentification, de mise à jour de profil et de navigation par rôle ;
- surface d'attaque visible liée à Firebase / Firestore ;
- gestion des jetons et comportement de session observé dans les éléments d'assistance fournis.

Hors périmètre :

- audit exhaustif du code source backend ;
- revue complète de l'infrastructure réseau et des services cloud ;
- analyse forensique complète des journaux de production ;
- tests intrusifs de charge à grande échelle ou d'indisponibilité.

3. Méthodologie d'évaluation AppSec

L'approche retenue est une démarche AppSec hybride combinant analyse fonctionnelle, manipulation des flux applicatifs, contrôle manuel des entrées, observation du comportement de l'API et raisonnement architectural. Les tests ont été orientés vers les familles OWASP les plus pertinentes pour une application de gestion à rôles multiples.

Étape	Objectif	Activités réalisées
Cartographie	Identifier les zones exposées	Analyse des rôles, endpoints, flux d'inscription / login / profil, zones de confiance et interactions cloud.
Manipulation HTTP	Tester les contrôles serveur	Interception et modification des requêtes via Burp Suite, ajout de champs sensibles, réémission des requêtes.
Tests d'abus	Vérifier la robustesse	Brute force, payloads volumineux, entrées inattendues et observation des codes de réponse.
Analyse session	Évaluer le modèle d'authentification	Observation des artefacts de session, décodage de jeton, examen du contenu des claims et des risques de réutilisation.
Corrélation risque	Qualifier l'impact métier	Lien entre constats techniques, privilèges accessibles, exposition des données et continuité de service.

4. Vue d'architecture et zones de confiance

L'architecture transmise met en évidence trois zones de confiance distinctes : la zone utilisateurs, la zone cloud hébergeant le frontend et l'API, puis la zone Google Cloud hébergeant Firestore. Les flux critiques convergent vers l'API backend, qui constitue le point de décision principal en matière d'authentification, d'autorisation, de validation et de journalisation.

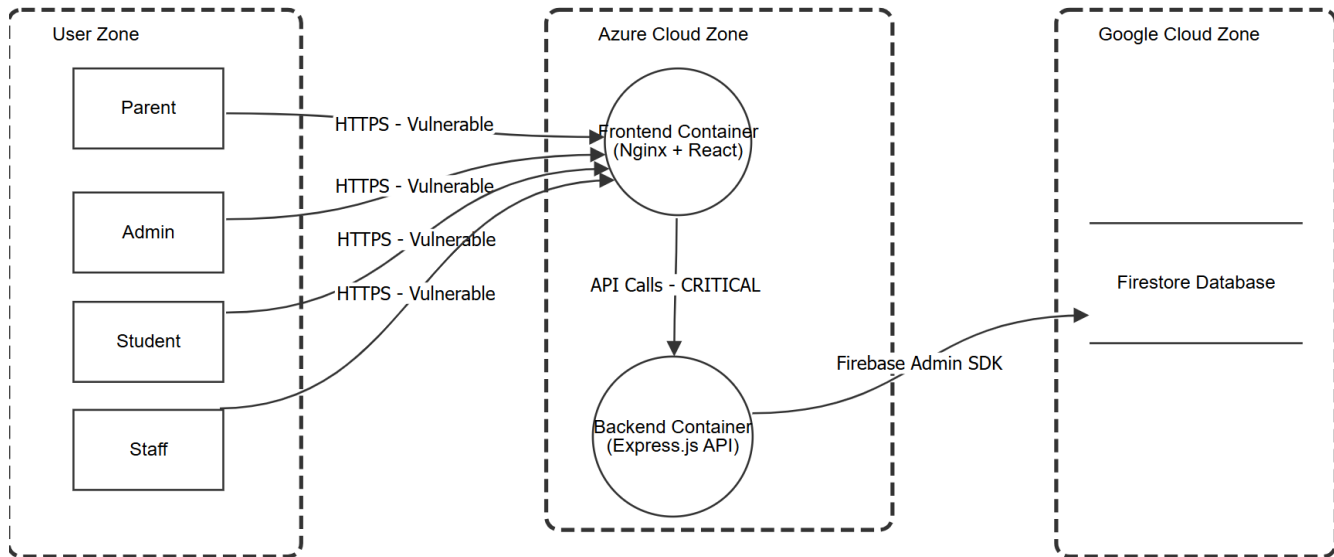


Figure 1 - Architecture logique avec trust boundaries et flux critiques

Lecture sécurité de l'architecture :

- les utilisateurs consomment l'application depuis une zone non fiable ;
- le frontend ne doit porter aucune décision de sécurité critique ;
- l'API backend doit être la seule source de vérité pour les rôles, permissions, états de compte et autorisations métier ;
- Firestore ne doit pas être accessible d'une manière contournant les règles métier ;
- le cycle de vie des sessions doit être géré côté serveur de manière cohérente avec les niveaux de privilège.

5. Analyse des causes racines

Au-delà des vulnérabilités unitaires, plusieurs causes racines transverses expliquent la majorité des constats. Leur mise en évidence est essentielle car corriger un seul endpoint sans traiter le défaut systémique laisserait d'autres vecteurs ouverts.

Cause racine	Manifestation observée	Effet sécurité
Confiance excessive accordée au client	Champs role / isActive transmis dans les requêtes	Élévation de privilèges, contournement des workflows métier
Contrôles d'accès insuffisants côté backend	Vérifications implicites ou absentes sur les opérations sensibles	Broken Access Control systémique
Gestion de session perfectible	Jeton JWT exposé et réutilisable, claims sensibles décodables	Prise de contrôle de compte, persistance illicite de session
Validation des entrées incomplète	Payloads volumineux ou inattendus provoquant des comportements non maîtrisés	Consommation abusive de ressources, erreurs 500
Détection et journalisation insuffisantes	Absence apparente d'alertes sur les tentatives anormales	Réactivité réduite face à l'exploitation ou à l'abus

6. Détail des constats techniques

F-01 - Privilege Escalation via Role Manipulation

Sévérité	Critique
Référentiel	OWASP A01: Broken Access Control - CWE-269 / CWE-284
Composant affecté	Endpoint de mise à jour du profil ou endpoint équivalent exposé à un utilisateur authentifié.
Qualification	Constat confirmé sur la base des éléments disponibles

Description.

Un utilisateur authentifié standard peut modifier son niveau de privilège en transmettant une valeur arbitraire dans le champ role lors de la mise à jour du profil. Le backend n'applique pas correctement les règles d'autorisation sur cet attribut sensible. Cette faiblesse compromet directement le modèle RBAC de l'application.

Scénario de reproduction synthétique.

- s'authentifier avec un compte standard ;
- intercepter la requête de mise à jour du profil ;
- modifier le champ role par une valeur privilégiée, par exemple admin ;
- rejouer la requête et vérifier l'accès à des fonctionnalités réservées à l'administration.

Impact métier et sécurité.

Cette vulnérabilité permet potentiellement la prise de contrôle logique de la plateforme : accès aux fonctions d'administration, modification d'autres comptes, consultation ou altération de données sensibles, et possibilité d'élargir ensuite la compromission.

Correctifs recommandés.

- supprimer totalement la possibilité pour un utilisateur standard de modifier role, isActive ou tout attribut de sécurité ;
- implémenter une liste blanche stricte des champs modifiables dans le DTO / schéma de validation ;
- contrôler l'autorisation côté serveur pour chaque opération sensible ;
- journaliser et alerter toute tentative de modification d'attributs critiques.

Validation attendue après correction.

Rejouer la même requête après correction et confirmer que le serveur ignore ou rejette le champ role. Vérifier l'absence de changement effectif des privilèges et la présence d'une trace applicative.

Vulnérabilité de session : JWT exposé et structure faible

Impact

Un jeton JWT (JSON Web Token) a été trouvé en clair dans les images partagées, notamment dans le fichier 1.jpg. Ce jeton contient des informations critiques encodées en Base64, facilement décodables. L'exposition d'un tel jeton permet à un attaquant de :

Prendre le contrôle du compte de l'utilisateur associé (test4 test4 avec le rôle Admin Principal) sans avoir besoin d'identifiants.

Accéder à l'ensemble des fonctionnalités d'administration, y compris la gestion des utilisateurs, des rôles, des paiements, et des données sensibles de l'établissement.

Contourner l'authentification et maintenir une session active de manière illicite.

Élever ses privilèges si le jeton contient des champs modifiables ou mal signés.

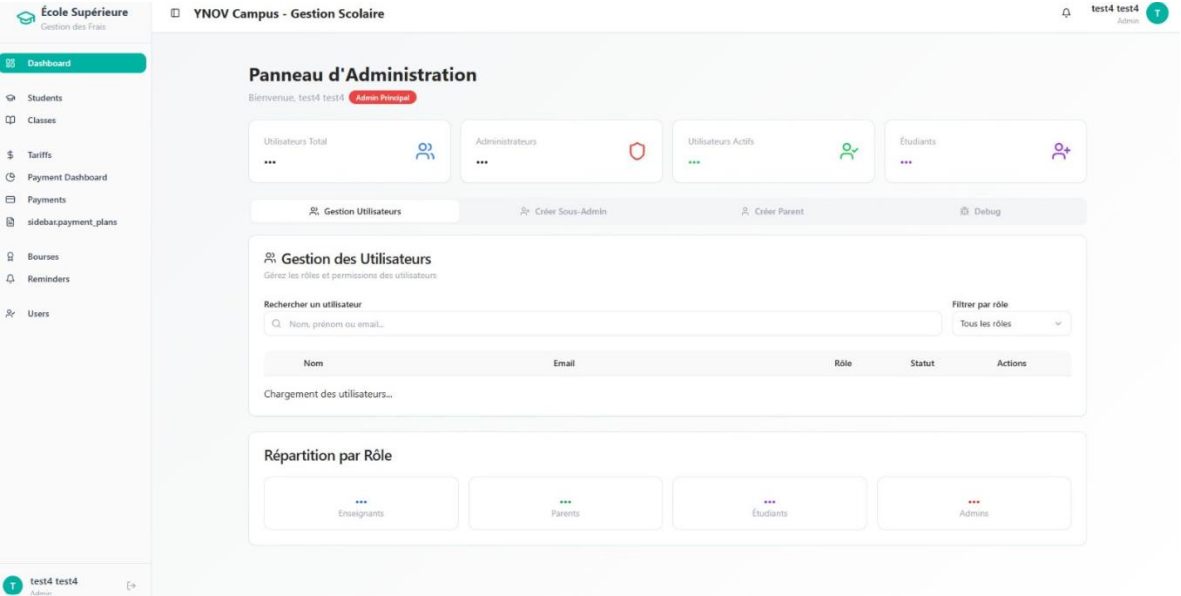
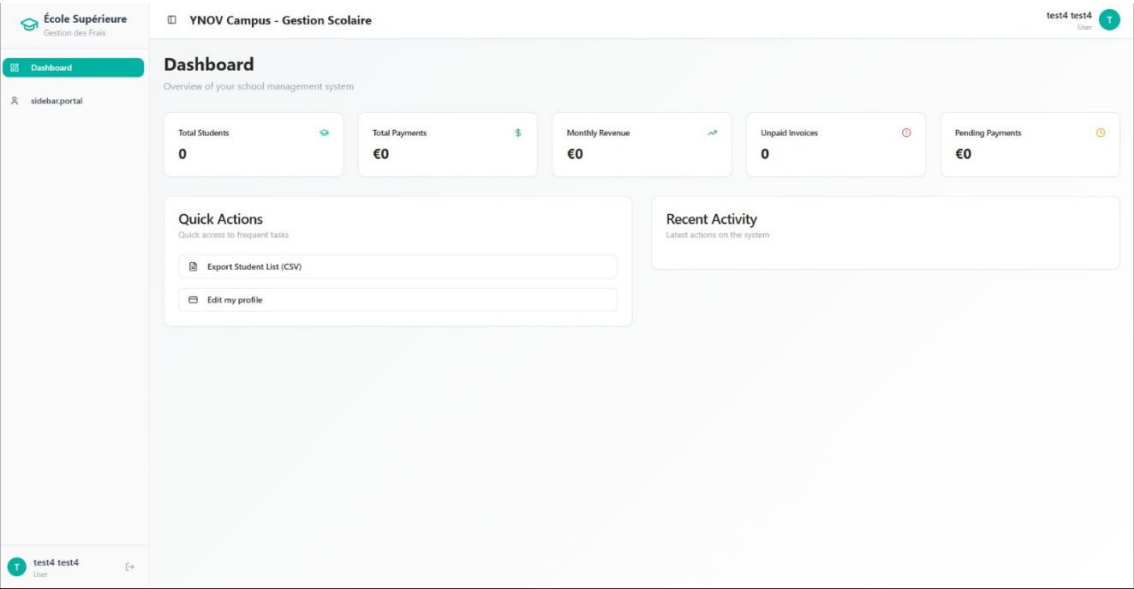
De plus, la structure du jeton semble incomplète ou mal formée (présence de caractères anormaux comme #wQoFi9AKUaujoIO3N40w), ce qui pourrait indiquer une faiblesse dans la génération ou la validation des jetons, augmentant le risque de falsification.

```
{ "id": "SwQoF9AKUaujO3N40", "email": "test4@gmail.com", "nom": "test4", "prenom": "test4", "role": "admin", "isActive": true }
```

Recommendation

- Ne jamais exposer de jetons d'authentification dans des fichiers accessibles publiquement (images, logs, code source). Les jetons doivent être stockés de manière sécurisée (HTTP-only cookies, stockage mémoire sécurisé).
- Révoquer immédiatement le jeton exposé et invalider toutes les sessions actives pour l'utilisateur concerné.
- Renforcer la politique de signature des JWT :
- Utiliser une clé secrète forte et unique.
- S'assurer que l'algorithme de signature n'est pas modifiable (alg: none doit être interdit).
- Ajouter des claims de sécurité comme exp (expiration courte), iat, et jti (identifiant unique de jeton).
- Mettre en place une validation côté serveur pour chaque requête authentifiée, avec vérification de la signature, de l'expiration, et des permissions.
- Auditer les accès pour détecter toute utilisation anormale du jeton compromis.
- Former les développeurs et administrateurs aux bonnes pratiques de gestion des sessions et des secrets.

Résultat final de vulnérabilité (broken access control)



F-02 - Privilege Escalation in Registration Flow

Sévérité	Critique
----------	----------

Référentiel	OWASP A01: Broken Access Control - CWE-639 / CWE-915
Composant affecté	Endpoint d'inscription / création de compte.
Qualification	Constat confirmé historiquement ; indices de correction partielle observés, revalidation complète recommandée

Description.

Le flux d'inscription acceptait des champs sensibles tels que role et isActive depuis le client. Un attaquant pouvait ainsi tenter d'influencer directement le niveau de privilège ou l'état du compte lors de la création. Les éléments fournis montrent désormais un workflow d'inscription renvoyant un statut pending, ce qui va dans le sens d'un durcissement partiel. Toutefois, la preuve disponible ne suffit pas à elle seule à démontrer que toute valeur arbitraire de role ou isActive est explicitement rejetée côté serveur.

Scénario de reproduction synthétique.

- intercepter ou reproduire la requête d'inscription ;
- ajouter des champs sensibles non prévus au corps de requête ;
- soumettre la requête avec des valeurs privilégiées ;
- vérifier si le compte créé reçoit des privilèges anormaux ou un état actif non autorisé.

Impact métier et sécurité.

Si le serveur accepte ces attributs, un attaquant peut créer un compte à privilèges élevés sans compromettre un compte existant. Dans un contexte réel, cela constitue un chemin direct de compromission administrative.

Correctifs recommandés.

- ignorer côté serveur tous les attributs de sécurité transmis par le client pendant l'inscription ;
- assigner les rôles exclusivement selon une logique métier backend ;
- forcer un état initial contrôlé, par exemple pending, avec validation par un administrateur ;
- ajouter des tests de non-régression garantissant le rejet systématique des champs sensibles.

Validation attendue après correction.

Exécuter à nouveau le scénario d'inscription avec des valeurs privilégiées forcées et confirmer que le backend les ignore entièrement. Vérifier que le compte reste dans l'état métier prévu et qu'aucun privilège supplémentaire n'est accordé.

F-03 - Missing Rate Limiting on Authentication Endpoint

Sévérité	Élevé
Référentiel	OWASP A07 / A04 selon implémentation - CWE-307
Composant affecté	Endpoint d'authentification (login).
Qualification	Constat confirmé sur la base des éléments disponibles

Description.

Le point d'authentification ne limite pas suffisamment les tentatives de connexion. L'absence de rate limiting, de verrouillage temporaire ou de challenge adaptatif facilite les attaques par brute force et credential stuffing.

Scénario de reproduction synthétique.

- envoyer de multiples tentatives de connexion avec des mots de passe différents ;
- répéter les requêtes en série ou en parallèle ;
- constater l'absence de blocage, de temporisation ou de CAPTCHA.

Request	Payload	Status code	Response received	Error	Timeout	Length	Comment
0		401	399			995	
1	123456	401	351			995	
2	123456789	401	384			995	
3	azerty	401	462			995	
4	1234561	401	429			995	
5	qwerty	401	607			995	
6	marseille	401	318			995	
7	000000	401	333			995	
8	1234567891	401	346			995	
9	doudou	401	346			995	
10	12345	401	360			995	
11	loulou	401	431			995	
12	123	401	353			995	
13	password	401	359			995	
14	azertyuiop	401	433			995	
15	12345678	401	424			995	
16	soleil	401	366			995	
17	chouchou	401	320			995	
18	1234	401	340			995	
19	1234567	401	428			995	
20	123123	401	274			995	
21	123451	401	374			995	
22	bonjour	401	317			995	
23	111111	401	471			995	
24	nicolas	401	401			995	
25	jetaime	401	279			995	
26	coucou	401	363			995	
27	motdepasse	401	279			995	
28	1234567890	401	303			995	

Request

```

1 POST /gestionadministration/us-centra11/api/v1/auth/login HTTP/2
2 Host: gestionadministration-centra11-48107264.francecentral.azurecontainer.io

```

Response

```

1 HTTP/2 401 Unauthorized
2 Server: nginx/1.16.5

```

Impact métier et sécurité.

Cette faiblesse accroît fortement le risque de compromission de comptes, surtout en cas de mots de passe faibles ou réutilisés. Elle augmente aussi la surface d'exposition face à des attaques automatisées à grande échelle.

Correctifs recommandés.

- mettre en place un rate limiting par IP et par compte ;
- ajouter un verrouillage temporaire progressif après plusieurs échecs ;
- journaliser les séquences d'échecs et générer des alertes ;
- prévoir un challenge adaptatif ou un CAPTCHA au-delà d'un seuil.

Validation attendue après correction.

Réaliser un nouveau test de tentatives répétées et confirmer qu'un seuil déclenche le blocage, la temporisation ou le challenge prévu.

F-04 - Missing Input Size Validation

Sévérité	Moyen
Référentiel	OWASP A04: Insecure Design / A05 - CWE-20
Composant affecté	Flux d'authentification et validation des entrées.
Qualification	Constat confirmé sur la base des éléments disponibles

Description.

Le champ password accepte des chaînes volumineuses sans politique claire de longueur maximale. Même si le serveur retourne ensuite une erreur, le traitement de ces entrées peut consommer inutilement du CPU et de la mémoire, notamment si elles sont combinées à des attaques répétées.

Scénario de reproduction synthétique.

- soumettre une requête d'authentification avec un mot de passe anormalement long ;
- observer la réponse serveur et le comportement applicatif ;
- répéter l'opération à grande fréquence pour évaluer la dégradation.

Impact métier et sécurité.

Isolé, l'impact est modéré. En revanche, combiné à l'absence de rate limiting, ce défaut peut contribuer à une consommation abusive des ressources et à un début de déni de service applicatif.

Correctifs recommandés.

- définir des longueurs maximales côté backend pour tous les champs sensibles ;
- rejeter précocement les payloads trop volumineux ;
- appliquer aussi des limites au niveau reverse proxy / gateway ;
- uniformiser les messages d'erreur sans divulgation technique.

Validation attendue après correction.

Rejouer le test avec des chaînes dépassant le seuil autorisé et confirmer un rejet immédiat, stable et peu coûteux en ressources.

F-05 - Weak Server-Side Security Architecture

Sévérité	Élevé
Référentiel	OWASP A04: Insecure Design / A01: Broken Access Control
Composant affecté	Architecture applicative frontend/backend et logique d'autorisation.
Qualification	Constat confirmé sur la base des éléments disponibles

Description.

Plusieurs constats montrent que des décisions de sécurité critiques semblent dépendre du client : restrictions visibles dans l'interface, confiance excessive dans les champs envoyés par l'utilisateur, absence d'application systématique des règles métier au niveau API. Ce défaut n'est pas un bug isolé mais un problème d'architecture de sécurité.

Scénario de reproduction synthétique.

- identifier les restrictions visibles uniquement dans l'interface utilisateur ;
- contourner ces restrictions en forgeant directement des requêtes HTTP ;
- vérifier si le backend applique ou non des validations indépendantes du frontend.

Impact métier et sécurité.

Cette faiblesse amplifie toutes les autres vulnérabilités. Dès qu'un attaquant peut modifier une requête, la sécurité affichée côté interface devient caduque. Le risque est systémique et affecte potentiellement plusieurs modules métier.

Correctifs recommandés.

- centraliser l'autorisation dans le backend ;
- mettre en œuvre une couche de contrôle d'accès explicite par rôle et permission ;
- évaluer les permissions à partir du contexte authentifié serveur et non des données clientes ;
- ajouter des tests automatisés de sécurité sur les règles d'autorisation.

Validation attendue après correction.

Vérifier endpoint par endpoint qu'un utilisateur non autorisé reçoit un refus côté serveur, indépendamment de l'état de l'interface.

F-06 - Firebase Attack Surface and Error Handling

Sévérité	Moyen / point d'attention
Référentiel	Observation complémentaire - nécessite validation technique ciblée
Composant affecté	Endpoints et intégration Firebase / Firestore / Cloud Functions.
Qualification	Constat confirmé sur la base des éléments disponibles

Description.

Des payloads inattendus envoyés au login provoquent des erreurs 500. Cela ne confirme pas une injection NoSQL exploitable, mais montre une gestion insuffisante des erreurs et signale une surface d'attaque à examiner plus en profondeur au niveau de Firebase, des règles Firestore, de l'API REST ou des Cloud Functions.

Scénario de reproduction synthétique.

- tester des payloads inattendus ou de type opérateur NoSQL ;
- observer les réponses 500 Internal Server Error ;
- rechercher les éléments de configuration exposés côté client ;
- vérifier les règles de sécurité et les accès directs aux services.

Request	Response
<pre> 1 POST /gestionadministration/us-central1/api/v1/auth/login HTTP/2 2 Host: gestion-admin-frontend.happybay-48397494.francecentral.azurecontainerapps.io 3 Content-Length: 61 4 Sec-CH-UA-Platform: "Windows" 5 Accept-Language: en-US,en;q=0.9 6 Accept: application/json, text/plain, */* 7 Sec-CH-UA: "Chromium";v="145", "Not:A-Brand";v="99" 8 Content-Type: application/json 9 Sec-CH-UA-Mobile: ?0 10 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36 11 Origin: https://gestion-admin-frontend.happybay-48397494.francecentral.azurecontainerapps.io 12 Sec-Fetch-Site: same-origin 13 Sec-Fetch-Mode: cors 14 Sec-Fetch-Dest: empty 15 Referer: https://gestion-admin-frontend.happybay-48397494.francecentral.azurecontainerapps.io/login 16 Accept-Encoding: gzip, deflate, br 17 Priority: u=1, i 18 19 { 20 "email": "test@test.com", 21 "password": { 22 "tgt": "" 23 } 24 }</pre>	<pre> 1 HTTP/2 500 Internal Server Error 2 Server: nginx/1.29.5 3 Date: Tue, 17 Mar 2026 21:53:00 GMT 4 Content-Type: application/json; charset=utf-8 5 Content-Length: 43 6 Content-Security-Policy: default-src 'self';base-uri 'self';font-src 'self' https: data:;form-action 'self';frame-ancestors 'self';img-src 'self' data:object-src 'none';script-src 'self';script-src-attr 'none';style-src 'self' https: 'unsafe-inline' upgrade-insecure-requests 7 Cross-Origin-Opener-Policy: same-origin 8 Cross-Origin-Resource-Policy: same-origin 9 Origin-Agent-Cluster: ?1 10 Referrer-Policy: no-referrer 11 Strict-Transport-Security: max-age=15552000; includeSubDomains 12 X-Content-Type-Options: nosniff 13 X-Dns-Prefetch-Control: off 14 X-Download-Options: noopen 15 X-Frame-Options: SAMEORIGIN 16 X-Permitted-Cross-Domain-Policies: none 17 X-Xss-Protection: 0 18 Vary: Origin 19 X-Powered-By: Express 20 Access-Control-Allow-Credentials: true 21 Etag: W/"2b-GJMD1j+aqRqXhN5s6UqxcOX7gs" 22 23 { 24 "message": "Erreur serveur", 25 "status": false 26 }</pre>

Impact métier et sécurité.

À ce stade, l'impact confirmé porte surtout sur la robustesse et la fuite potentielle d'information via une mauvaise gestion d'erreur. Si des mauvaises configurations cloud existent, l'impact pourrait devenir critique ; ce point doit donc être audité avec prudence et rigueur.

Correctifs recommandés.

- uniformiser la gestion des erreurs et éviter les réponses 500 sur simple entrée invalide ;
- auditer les règles Firestore et les accès Cloud Functions ;
- vérifier l'exposition de la configuration Firebase côté client ;
- réaliser une revue dédiée des secrets, permissions de service account et surfaces d'appel.

Validation attendue après correction.

Après correction, rejouer les mêmes payloads et confirmer des réponses gérées, génériques et sans fuite d'information technique.

F-07 - JWT exposé et structure de session à renforcer

Sévérité	Critique
Référentiel	OWASP A02: Cryptographic Failures / A07: Identification and Authentication Failures / CWE-522 / CWE-613
Composant affecté	Gestion de session, artefacts d'authentification et validation des requêtes authentifiées.
Qualification	Constat confirmé sur la base des éléments disponibles

Description.

Un jeton JWT (JSON Web Token) de session a été observé en clair dans des éléments d'assistance transmis dans le cadre des tests. Ce jeton, décodable en Base64, exposait des informations sensibles de contexte utilisateur et de privilège. L'exposition d'un tel artefact permet potentiellement à un attaquant de réutiliser la session associée sans connaître les identifiants du compte concerné. La structure observée du jeton et certains caractères anormaux visibles imposent également une revue de la génération, de la signature et de la validation des JWT afin d'écarter tout risque de falsification, de vérification insuffisante ou d'acceptation de jetons mal formés.

Scénario de reproduction synthétique.

- récupérer le jeton exposé dans un artefact non protégé ;
- le décoder pour identifier les claims de rôle ou d'utilisateur ;
- le réinjecter dans une requête authentifiée ou une session navigateur ;
- vérifier l'accès obtenu aux fonctionnalités privilégiées tant que le jeton reste valide.

Impact métier et sécurité.

L'impact potentiel est critique. L'exposition d'un tel jeton peut permettre à un attaquant de :

- prendre le contrôle du compte associé sans avoir besoin des identifiants ;
- accéder aux fonctionnalités d'administration, y compris la gestion des utilisateurs, des rôles, des paiements et d'autres données sensibles ;

- contourner l'authentification et maintenir illicitement une session valide tant que le jeton n'est pas révoqué ou expiré ;
- élever ses privilèges si le contenu du jeton est trop permissif, mal signé ou insuffisamment vérifié côté serveur.

Correctifs recommandés.

- ne jamais exposer de jetons d'authentification dans des fichiers accessibles publiquement, des logs, des captures ou du code source ;
- révoquer immédiatement le jeton exposé et invalider toutes les sessions actives du compte concerné ;
- stocker les jetons de manière sécurisée, de préférence dans des cookies HttpOnly, Secure et SameSite, ou dans un stockage mémoire strictement contrôlé ;
- utiliser une clé de signature forte et unique et interdire toute acceptation d'algorithme non maîtrisé, notamment alg=none ;
- ajouter et contrôler les claims de sécurité exp, iat, nbf et jti, avec une durée de vie courte pour les jetons d'accès ;
- mettre en place une validation côté serveur à chaque requête authentifiée avec vérification de la signature, de l'expiration, de la révocation et des permissions effectives ;
- auditer les accès et les journaux afin de détecter toute utilisation anormale du jeton compromis ;
- former les développeurs et les administrateurs aux bonnes pratiques de gestion des sessions, des secrets et des artefacts de support.

Validation attendue après correction.

Après remédiation, vérifier qu'un jeton révoqué ne donne plus accès à l'application, que les jetons expirent rapidement, qu'ils ne sont plus visibles dans les artefacts fonctionnels, que la rotation ou le renouvellement de session est correctement géré, et que toute requête authentifiée dépend d'une validation serveur stricte de la signature, de l'expiration et des permissions.

F-09 - Divulcation d'informations techniques : exposition de la version du serveur web

Sévérité	Référentiel	Composant affecté	Statut
Faible à moyen	OWASP A05 / Information Disclosure	Frontend servi par Nginx	Confirmé

Description

Les vérifications passives réalisées sur l'application ont montré que le serveur web expose sa version exacte dans l'en-tête HTTP Server. La réponse observée retourne la valeur nginx/1.29.5, ce qui constitue une fuite d'information technique exploitable durant la phase de reconnaissance.

En complément, l'appel d'une route inexistante et d'un chemin typiquement utilisé pour le diagnostic tel que /phpinfo.php renvoie le même contenu frontend avec un code 200 OK. Ce comportement indique un mécanisme de fallback applicatif de type SPA, mais ne permet pas de confirmer l'utilisation de PHP.

Contexte d'observation

- La requête HEAD sur le domaine principal renvoie l'en-tête Server avec la version complète de Nginx.
- Une route inexistante retourne 200 OK et sert index.html au lieu d'une réponse 404 explicite.
- Le chemin /phpinfo.php renvoie la même page frontend, sans exposition d'une page phpinfo réelle.
- Aucune preuve de version PHP n'a été observée dans les en-têtes testés à ce stade.

```
PS C:\Users\lourini.a> curl.exe -I https://gestion-admin-frontend.happybay-48397494.francecentral.azurecontainerapps.io
HTTP/1.1 200 OK
server: nginx/1.29.5
date: Sun, 29 Mar 2026 14:56:09 GMT
content-type: text/html
content-length: 1159
last-modified: Fri, 06 Mar 2026 20:51:04 GMT
etag: "69ab3e38-487"
accept-ranges: bytes

PS C:\Users\lourini.a>
```

Figure 1 - Preuve d'observation : l'en-tête HTTP expose la version exacte du serveur web.

Exemple de payload / requête

Commande de vérification passive utilisée :

```
curl.exe -I https://gestion-admin-frontend.happybay-48397494.francecentral.azurecontainerapps.io
```

Extrait de réponse observé :

```
HTTP/1.1 200 OK
server: nginx/1.29.5
content-type: text/html
```

Impact

- Facilitation de la phase de fingerprinting et de reconnaissance technique.

- Aide au ciblage de vulnérabilités connues liées à une version spécifique de Nginx.
- Réduction de l'effort nécessaire pour orienter un attaquant vers des vecteurs adaptés à l'environnement d'exécution.
- Révélation inutile d'informations techniques qui n'apportent aucune valeur fonctionnelle à l'utilisateur final.

Analyse

Le risque direct reste inférieur à une faille d'autorisation ou de session, mais cette exposition améliore la qualité du renseignement disponible pour un attaquant. Dans une démarche de défense en profondeur, les versions exactes des composants d'infrastructure ne doivent pas être divulguées publiquement lorsqu'elles ne sont pas strictement nécessaires.

Les tests effectués montrent également que certaines routes inexistantes renvoient la page frontend avec un code 200 OK. Cela semble cohérent avec un routage client de type Single Page Application, mais ce choix doit être maîtrisé afin d'éviter la confusion entre routes applicatives valides et ressources réellement absentes.

Cause racine

- Configuration Nginx laissant apparaître la version exacte du serveur dans l'en-tête Server.
- Absence de politique de réduction des en-têtes techniques exposés publiquement.
- Configuration de fallback frontend qui masque les erreurs 404 classiques sur certaines routes.

Correctif recommandé

- Désactiver l'exposition de la version Nginx avec la directive `server_tokens off;`.
- Supprimer ou réduire au strict nécessaire les en-têtes techniques non indispensables.
- Vérifier qu'aucune page de debug ou de diagnostic n'est accessible en production.
- Revoir la gestion des routes inexistantes si une réponse 404 explicite est requise pour certaines ressources non applicatives.
- Intégrer un contrôle de non-régression dans les tests de déploiement afin de vérifier l'absence d'exposition des versions d'infrastructure.

Exemple de correctif

```
http {  
    server_tokens off;  
}
```

Validation après correction

- Relancer la commande HEAD sur le domaine principal et vérifier que la valeur observée n'expose plus la version exacte du serveur.
- Contrôler que l'en-tête Server est réduit à nginx ou supprimé selon la capacité de la plateforme.
- Confirmer qu'aucun endpoint de debug ne révèle d'informations complémentaires sur l'environnement d'exécution.

Texte court à intégrer dans la matrice de synthèse

ID	Constat	Sévérité	Statut	Action attendue
F-09	Divulgence de la version Nginx via l'en-tête Server	Faible à moyen	Confirmé	Masquage des informations de version

7. Remédiation, validation et re-test

Pour transformer ce rapport en livrable PFE solide, il est essentiel de documenter non seulement les vulnérabilités découvertes, mais aussi la logique de correction et la preuve que la surface d'attaque a réellement diminué. La section suivante propose une structure de validation directement réutilisable après implémentation des correctifs.

7.1. Modèle de validation par vulnérabilité

ID	Cause racine	Correctif appliqué	Méthode de vérification	Résultat post-correction	Statut
F-01	Confiance côté client sur role	Whitelist backend + contrôle d'autorisation	Rejouer la même requête via Burp	Le champ role est ignoré ou rejeté	À valider

7.2. Tableau de synthèse avant / après

ID	Vulnérabilité	Gravité initiale	Correctif attendu	Validation	Statut cible
F-01	Escalade via role	Critique	Suppression des champs sensibles modifiables + contrôle serveur	Retest Burp concluant	Corrigée
F-02	Inscription avec privilèges	Critique	Attribution des rôles uniquement côté serveur	Compte créé sans privilège anormal	Corrigée
F-03	Absence de rate limiting	Élevé	Limitation des tentatives + journalisation	Blocage après seuil	Corrigée
F-04	Taille d'entrée non limitée	Moyen	Validation de longueur + rejet précoce	Payload long refusé	Corrigée
F-05	Sécurité dépendante du frontend	Élevé	Contrôles déplacés côté backend	Contournement client inefficace	Corrigée
F-06	Erreurs 500 / surface cloud	Moyen	Validation stricte + revue cloud	Réponses maîtrisées	Partiellement corrigée
F-07	JWT exposé / session faible	Critique	Révocation, rotation, HttpOnly, validation serveur	Jeton compromis inutilisable	Corrigée

7.3. Stratégie de re-test recommandée

- rejouer exactement les mêmes scénarios d'attaque qu'avant remédiation ;
- conserver les preuves avant / après (requête, réponse, capture ou log expurgé) ;
- vérifier que le correctif agit côté serveur et pas uniquement côté interface ;

- s'assurer qu'aucun effet de bord n'introduit une nouvelle faiblesse ;
- conclure chaque finding par un statut : corrigée, partiellement corrigée, non corrigée, à surveiller.

8. Contrôles de sécurité à mettre en place

Les correctifs unitaires doivent être complétés par des contrôles de sécurité transverses afin de réduire durablement le risque de réapparition des mêmes défauts.

Contrôle	Objectif	Priorité
RBAC côté serveur	Toutes les décisions de privilège doivent être évaluées au backend à partir du contexte authentifié.	Critique
Validation stricte des DTO	Limiter les champs acceptés et leurs types / formats / tailles.	Critique
Gestion sécurisée des sessions	Cookies HttpOnly, rotation, expiration courte, révocation, contrôle jti.	Critique
Rate limiting & lockout	Réduire brute force, credential stuffing et abus de ressources.	Élevée
Journalisation sensible	Tracer changement de rôle, activation, connexion, erreurs 500, activités admin.	Élevée
Supervision / alerting	Détecter rapidement des usages anormaux, réutilisations de session ou pics d'échec.	Élevée
Tests AppSec CI/CD	Éviter la réintroduction des mêmes vulnérabilités lors des évolutions.	Moyenne
Revue cloud Firebase	Valider les règles, secrets, accès inter-services et surfaces exposées.	Élevée

9. Mapping des constats avec l'OWASP Top 10

ID	Catégorie OWASP	Justification
F-01	A01 Broken Access Control	Le client influence directement un attribut de privilège.
F-02	A01 Broken Access Control	Le flux de création de compte accepte des champs de sécurité non autorisés.
F-03	A07 Identification and Authentication Failures	Le login ne freine pas les tentatives répétées.
F-04	A04 Insecure Design / A05 Security Misconfiguration	Absence de bornes d'entrée et comportement peu robuste face aux abus.
F-05	A04 Insecure Design	La sécurité dépend excessivement de l'interface et non du backend.
F-06	A05 Security Misconfiguration / A09 Logging & Monitoring Failures	Gestion d'erreurs perfectible et configuration cloud à revoir.

F-07	A07 Identification and Authentication Failures / A02 Cryptographic Failures	Exposition de session et nécessité de renforcer la politique JWT.
------	---	---

10. Feuille de route de remédiation priorisée

Horizon	Actions prioritaires	Objectif sécurité	Responsables indicatifs
0-7 jours	Révoquer les jetons exposés, invalider les sessions, bloquer role / isActive côté serveur.	Supprimer les vecteurs de compromission immédiats.	Backend / sécurité
1-2 semaines	Déployer rate limiting, lockout, tailles maximales et whitelist de champs.	Réduire brute force et abus de ressources.	Backend / DevOps
2-4 semaines	Centraliser les contrôles d'autorisation et ajouter des tests automatiques.	Installer une défense en profondeur durable.	Backend / QA sécurité
4-6 semaines	Auditer Firebase, Firestore, Cloud Functions et la gestion des secrets.	Réduire les contournements hors logique métier.	Cloud / sécurité applicative
6-8 semaines	Mettre en place supervision, alerting et procédure de re-test sécurité.	Améliorer la détection et la validation continue.	SecOps / projet

11. Limites de l'étude et perspectives

Limites de l'étude.

- les tests ont été conduits sur un périmètre applicatif défini et à partir des éléments observables ;
- certaines hypothèses, notamment autour de Firebase et de la signature exacte des JWT, nécessitent une validation technique complémentaire ;
- l'absence de revue exhaustive du code source limite la capacité à conclure sur tous les chemins d'exécution.

Perspectives d'amélioration.

- intégrer des tests SAST / DAST dans le pipeline de développement ;
- mettre en place une revue de code sécurité systématique ;
- introduire MFA pour les profils sensibles ;
- déployer une gouvernance des secrets et des clés de signature ;
- ajouter des tableaux de bord de sécurité et des cas d'usage de détection.

12. Protection des données personnelles et conformité RGPD

12.1 Contexte et enjeux

L'application web de gestion administrative et des frais scolaires traite des données à caractère personnel relatives aux étudiants, parents, personnel administratif et utilisateurs internes. Conformément au RGPD, le traitement doit garantir la confidentialité, l'intégrité, la disponibilité, la minimisation des données et la traçabilité des traitements.

12.2 Cartographie des données personnelles

Les données traitées incluent les informations d'identité, les données d'authentification, les données financières et les journaux d'activité. Une classification par niveau de sensibilité doit être appliquée afin d'adapter les contrôles de sécurité.

12.3 Risques RGPD identifiés

Les vulnérabilités identifiées dans l'évaluation AppSec peuvent entraîner des violations de données personnelles, notamment via le Broken Access Control, l'exposition de JWT ou une journalisation insuffisante. Ces risques peuvent conduire à un accès non autorisé aux informations sensibles.

12.4 Mesures techniques de conformité RGPD

- Minimisation des données collectées.
- Chiffrement des données sensibles au repos.
- Utilisation obligatoire de HTTPS/TLS.
- Cookies sécurisés HttpOnly, Secure et SameSite.
- Gestion des droits utilisateurs : accès, rectification, suppression et export des données.
- Journalisation des accès aux données sensibles.

12.5 Privacy by Design et Privacy by Default

La protection des données doit être intégrée dès la conception de l'application : contrôles d'accès côté backend, séparation des rôles, limitation des privilèges et réduction de l'exposition des données personnelles.

12.6 Validation de conformité

Des tests doivent vérifier l'accès restreint aux données personnelles, la suppression complète d'un compte utilisateur, l'expiration correcte des sessions et la traçabilité des accès aux données.

12.7 Conclusion sécurité et conformité

L'intégration des principes RGPD renforce la posture de sécurité globale de l'application. Elle réduit les risques de fuite de données et aligne le projet avec les exigences réglementaires et les bonnes pratiques OWASP.

13. Conclusion générale

Les vulnérabilités identifiées démontrent que la priorité absolue de ce projet n'est pas seulement de corriger des bugs ponctuels, mais de rétablir une architecture de sécurité cohérente. Le backend doit redevenir la seule source de vérité pour les décisions d'autorisation, de validation et d'activation des comptes. La découverte additionnelle d'un jeton JWT exposé confirme que la sécurité des sessions doit être traitée au même niveau de priorité que les contrôles d'accès.

Du point de vue PFE, ce rapport met en évidence une démarche AppSec complète : identification des failles, analyse des causes racines, qualification du risque, proposition de remédiations concrètes, définition d'un plan de validation et projection vers un hardening durable. La valeur du projet sera encore renforcée en ajoutant, après implémentation, les preuves de re-test avant / après pour chaque finding critique.